Project Plan: The Verified Market Simulator Pipeline

Financial Computing & Algo Trading Section

Spring 2026

1 Mission & Architecture

The goal is to build a high-performance, purely functional **Market Simulation Pipeline**. Unlike monolithic simulators, this system is designed as a modular set of tools that allow researchers to "Bring Your Own Data" (BYOD), ensuring compliance with LOBSTER data licensing.

1.1 The Pipeline Architecture

The system consists of two distinct, decoupled pipelines:

1. **The Data Pipeline (Preprocessing):** A local tool that ingests raw LOBSTER CSVs (provided by the user) and converts them into a high-efficiency, compressed binary format optimized for HPC storage and streaming.
2. **The Simulation Pipeline (Backtesting):** A purely functional OCaml engine running on the HPC cluster that streams the binary data, rebuilds the Limit Order Book (LOB) without mutation, and executes strategy logic.

2 Team Structure (Flexible)

We anticipate a team of 3–4 students. Rather than rigid silos, we propose a **Collaborative Swarm** model where all members participate in core development and verification, with rotating leads for specific components.

- **All Members:** Contribute to the core OCaml matching logic and Property-Based Testing (QCheck). This ensures everyone gains exposure to Functional Programming and Verification.
- **Rotation A (Infrastructure):** Focuses on the HPC Data Pipeline, binary file I/O, and parallelization.
- **Rotation B (Theory):** Focuses on formalizing invariants (e.g., Fair Matching) and proving them using formal methods (Coq/Cameleer).

3 Execution Timeline (6 Months)

Phase 1: The "Quick Start" & Data Foundation (Weeks 1–6)

Goal: Get a walking skeleton running immediately.

- **Team Task:** Complete *Real World OCaml* (Ch 1–5) and set up HPC environments.
- **The Data Pipeline:**

  - Build the *Python Preprocessor* (using Polars) to convert CSV → Binary locally.
  - Define the *Binary Schema*: Fixed-width integers for Price/Size/Time to avoid floating-point drift.

- **The Engine:**
  - Implement the `OrderBook` data structure (Map/Tree-based).
  - Implement a basic `CsvReader` (temporary) to allow immediate testing of the engine before the full Binary pipeline is ready.

Phase 2: The Core Matching Engine (Weeks 7–14)

Goal: A correct, functional LOB implementation.

- **Core Logic:** Implement the immutable `match` function.
  - Logic: $match(Book_t, Order_{new}) \rightarrow (Book_{t+1}, Trades, Updates)$.
- **Verification (All Hands):**
  - Define Invariants: *Best Bid* $<$ *Best Ask*, *Trade Price* $\in$ *[Limit, Market]*.
  - Implement **QCheck** generators to fuzz-test the engine with millions of random orders.
- **HPC Integration:** Connect the OCaml engine to the *Binary Data Stream* created in Phase 1.

Phase 3: Formal Verification & Visualization (Weeks 15–24)

Goal: Prove correctness and visualize drift.

- **The "Verifier" Workstream:**
  - Map the OCaml types to a Theorem Prover.
  - Attempt to prove that the matching logic satisfies *Fairness* (Price-Time Priority) and *Individual Rationality* [1].
- **The "Visualizer" Workstream:**
  - Build a tool to compare two state snapshots ($Book_t$ vs $Book'_t$) to visually debug any state drift.
- **Performance Tuning:** Profile the engine on the HPC cluster and optimize Garbage Collection (GC) for high throughput.

4 Why This Approach?

- **Flexibility:** By decoupling Data from Simulation, we avoid licensing issues and allow the engine to be open-sourced while data remains private.
- **Rigor:** Every team member gains exposure to Formal Verification, a highly sought-after skill in modern systems engineering (e.g., AWS, Jane Street).
- **Efficiency:** The binary pipeline ensures that the HPC cluster spends CPU cycles on *matching*, not text parsing.

References

[1] Suneel Sarswat and Abhishek Kr Singh. *Formally Verified Trades in Financial Markets*. In *Formal Methods and Software Engineering (ICFEM 2020)*, pages 217–232. Springer, 2020.